

CUSTOM PAYLOAD ENCODER OBFUSCATION FRAMEWORK

Malware Analysis and Signature Evasion

Domain: Cybersecurity

Technology Used: Kali Linux and Python 3

- Internship: Unified Mentor Internship – 2026
- **Developed by: P.Hariharan UMID03122572891**

CUSTOM PAYLOAD ENCODER OBFUSCATION FRAMEWORK

It demonstrates how **payloads are modified using encoding and obfuscation techniques.**

Key Features

- Supports multiple encoding techniques (Base64, XOR, ROT13)
- Implements **string obfuscation methods** to hide payload patterns.
- Allows **layered encoding & obfuscation** for stronger evasion testing.
- Includes **simulated signature-based detection module**
- Compares **original vs modified payload detection results**

PROJECT OBJECTIVES

- Study the limitations of **signature-based security systems**
- Understand how **payload transformation affects detection**
- Design a **modular payload encoding framework** in Python
- Implement multiple encoding techniques (Base64, XOR, ROT13)
- Develop **string obfuscation methods** to modify payload patterns
- Simulate a **basic detection engine** using pattern matching
- Perform **comparative analysis** between original and modified payloads
- Evaluate the **effectiveness of different evasion techniques**
- Demonstrate the need for **layered and behavioral security approaches**
- Build a **safe and controlled lab tool** for cybersecurity education

SYSTEM WORKFLOW / ARCHITECTURE

Payload Input

- User provides a test payload (text/script).

Encoding Selection

- Choose encoding method: **Base64, XOR, or ROT13.**

Encoding Selection

- Choose encoding method: **Base64, XOR, or ROT13.**

Apply Encoding Layer

- Convert the payload into encoded format.

String Obfuscation

- Apply techniques like string splitting, junk characters, and escape sequences.

Evasion Testing

- Run the original and modified payloads through **pattern-based detection.**

Result Comparison

- Check whether payload is **Detected or Bypassed.**

Output & Reporting

- Generate transformed payloads and display analysis results.

ENCODING TECHNIQUES

1. Base64 Encoding

- Converts payload into an ASCII text format
- Makes the payload unreadable to humans
- Commonly used to hide suspicious strings

2. XOR Encoding

- Uses a secret key to transform payload data
- Produces different output for different keys
- Simple encryption method often used for obfuscation

3. ROT13 Encoding

- Substitution cipher that shifts letters by 13 positions
- Changes readable text into another form
- Demonstrates basic text transformation techniques

4. Purpose of Encoding

- Change payload appearance
- Break known signatures and patterns
- Test effectiveness of detection systems

STRING OBFUSCATION AND EVASION TESTING

String Obfuscation

- Modifies readable payload strings to hide known patterns
- Techniques used:
 - Random character insertion
 - String splitting & concatenation
 - Escape sequence encoding
 - Reversible string transformations

Evasion Testing

- Simulates **signature-based detection** using keyword/pattern matching
- Tests both **original and obfuscated payloads**
- Measures whether payload is **Detected or Bypassed**

TOOLS AND TECHNOLOGIES

Programming Language

- **Python** – Used to develop the entire framework

Python Libraries

- **base64** – For Base64 encoding & decoding
- **random** – For generating random characters in obfuscation
- **itertools** – For string manipulation techniques
- **argparse** – For optional command-line interface

Development Environment

- Kali Linux / Windows (Lab setup)

RESULTS AND LEARNING OUTCOMES

```
(kali@kali)-[~]
$ cd Desktop

(kali@kali)-[~/Desktop]
$ mkdir payload_encoder_framework

(kali@kali)-[~/Desktop]
$ cd payload_encoder_framework

(kali@kali)-[~/Desktop/payload_encoder_framework]
$ touch main.py encoder.py obfuscator.py evasion_test.py report.py

(kali@kali)-[~/Desktop/payload_encoder_framework]
$ ls
encoder.py  evasion_test.py  main.py  obfuscator.py  report.py

(kali@kali)-[~/Desktop/payload_encoder_framework]
$ nano encoder.py

(kali@kali)-[~/Desktop/payload_encoder_framework]
$ nano obfuscator.py

(kali@kali)-[~/Desktop/payload_encoder_framework]
$ nano evasion_test.py

(kali@kali)-[~/Desktop/payload_encoder_framework]
$ nano report.py

(kali@kali)-[~/Desktop/payload_encoder_framework]
$ nano main.py

(kali@kali)-[~/Desktop/payload_encoder_framework]
$ python3 main.py
===== Payload Encoder & Obfuscator =====
Enter payload: payload

Choose Encoding:
1. Base64
2. ROT13
3. XOR
Choice: 1

Choose Obfuscation:
1. Random insertion
2. Split & concat
3. Escape sequence
Choice: 3
```

===== FINAL REPORT =====

Original Payload : payload

Modified Payload : \x63\x47\x46\x35\x62\x47\x39\x68\x5a\x41\x3d\x3d

Detection Status : NOT DETECTED

(kali@kali)-[~/Desktop/payload_encoder_framework]

\$ python3 main.py

===== Payload Encoder & Obfuscator =====

Enter payload: payload

Choose Encoding:

1. Base64
2. ROT13
3. XOR

Choice: 2

Choose Obfuscation:

1. Random insertion
2. Split & concat
3. Escape sequence

Choice: 2

===== FINAL REPORT =====

Original Payload : payload

Modified Payload : "cn" + "ly" + "bn" + "q"

Detection Status : NOT DETECTED

(kali@kali)-[~/Desktop/payload_encoder_framework]

\$ python3 main.py

===== Payload Encoder & Obfuscator =====

Enter payload: payload

Choose Encoding:

1. Base64
2. ROT13
3. XOR

Choice: 3

Enter XOR key: 1

Choose Obfuscation:

1. Random insertion
2. Split & concat
3. Escape sequence

Choice: 1

```
===== FINAL REPORT =====  
Original Payload : payload  
Modified Payload : AgPrHw]J^pPaUv  
Detection Status : NOT DETECTED  
=====
```

OUTPUT

CONCLUSION

- The project demonstrates how **encoding and obfuscation change payload signatures**.
- Shows the **limitations of signature-based detection systems**.
- Highlights the importance of **layered and advanced security mechanisms**.
- Provides a **practical lab framework** for cybersecurity learning and research.

THANK YOU

LINUX PRIVILEGE ESCALATION AUTOMATION TOOLKIT

Security Auditing Toolkit

Domain: Cybersecurity

Technology: Kali Linux, Python 3

Internship: Unified Mentor Internship – 2026

Developed by: P.Hariharan UMID03122572891

LINUX PRIVILEGE ESCALATION AUTOMATION KIT

A tool that automatically detects privilege escalation risks in Linux systems.

Key features:

- Automatically scans the Linux system for privilege escalation vulnerabilities.
- Performs **safe enumeration** without exploiting the system.
- Finds binaries running with elevated privileges that may be abused.
- Identifies **world-writable files and directories** that can be misused.
- Detects **root cron jobs** executing writable scripts.
- Checks for risky sudo rules such as **NOPASSWD** access.

PROJECT OBJECTIVES

- To automate the detection of Linux privilege escalation vulnerabilities
- To generate a clear and structured vulnerability report
- To reduce manual effort in security auditing
- To help beginners learn Linux security
- To identify system misconfigurations that may auditing lead to root access
- To improve system hardening and security awareness
- To provide a safe and ethical security assessment tool

TECHNOLOGY STACK

Component	Technology
Language	Python 3
Operating System	Kali Linux
Sudo Analyzer	sudo -l
Scripting	Bash
Capabilities Scanner	getcap

WORKFLOW / ARCHITECTURE

Step 1

- User runs the toolkit from the Linux terminal.

Step 2

- Toolkit gathers user details, OS version, kernel info, and permissions

Step 3

- • SUID/SGID Scanner
- Weak Permission Scanner
- Sudo Misconfiguration Check
- Cron Job Scanner
- Service Misconfiguration Scanner
- Kernel Vulnerability Check

Step 4

- All findings are analyzed and classified by risk level.

Step 5

- Toolkit generates a structured security report with mitigation steps.

Step 6

- End

IMPLEMENTATION

Step 1: (Set Up Environment)

Install Python and prepare a Linux testing machine (Kali/Ubuntu).

Step 2:(Develop Scanning Modules)

Create scripts to scan:

- SUID/SGID binaries
- Weak file permissions
- Sudo misconfigurations
- Cron jobs
- System services
- Kernel version

Step 3: (Integrate Modules)

Combine all scanning scripts into one automated toolkit.

Step 4: (Build Analysis Engine)

Filter Outputs and Classify Risks (High / Medium / Low)

Step 5: (Generate Report)

Automatically create a security report with findings and mitigation steps.

Step 6: (Testing & Validation)

Run toolkit in a virtual lab and verify results.

OUTPUT / RESULT

```
(kali@kali)-[~]
```

```
$ python3 privesc_toolkit.py
```

```
Linux Privilege Escalation Automation Toolkit
```

[+] System Information

```
User      : kali
```

```
Groups    : uid=1000(kali) gid=1000(kali) groups=1000(kali),4(adm),20(dialo  
ut),24(cdrom),25(floppy),27(sudo),29(audio),30(dip),44(video),46(plugdev),100  
(users),101(netdev),103(scanner),107(bluetooth),120(lpadmin),129(wireshark),1  
30(kaboxer),131(vboxsf)
```

```
Kernel    : 6.17.10+kali-amd64
```

```
OS        : PRETTY_NAME="Kali GNU/Linux Rolling"
```

[+] SUID Binary Scan

```
/usr/lib/polkit-1/polkit-agent-helper-1
```

```
/usr/lib/xorg/Xorg.wrap
```

```
/usr/lib/chromium/chrome-sandbox
```

```
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
```

```
/usr/lib/openssh/ssh-keysign
```

[+] Capability Scan

```
/usr/lib/x86_64-linux-gnu/gstreamer1.0/gstreamer-1.0/gst-ptp-helper cap_net_b  
ind_service,cap_net_admin,cap_sys_nice=ep
```

```
/usr/lib/nmap/nmap cap_net_bind_service,cap_net_admin,cap_net_raw=eip
```

```
/usr/bin/fping cap_net_raw=ep
```

```
/usr/bin/dumpcap cap_net_admin,cap_net_raw=eip
```

[+] World Writable Files

```
/run/user/1000/ICEauthority  
/run/user/1000/dconf/user  
/run/user/1000/pipewire-0-manager.lock  
/run/user/1000/pipewire-0.lock  
/run/user/1000/pulse/pid
```

[+] Cron Jobs

```
# /etc/crontab: system-wide crontab  
# Unlike any other crontab you don't have to run the `crontab`  
# command to install the new version when you edit this file  
# and files in /etc/cron.d. These files also have username fields,  
# that none of the other crontabs do.
```

```
SHELL=/bin/sh
```

```
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
```

```
# Example of job definition:
```

```
# .----- minute (0 - 59)  
# | .----- hour (0 - 23)  
# | | .----- day of month (1 - 31)  
# | | | .----- month (1 - 12) OR jan,feb,mar,apr ...  
# | | | | .----- day of week (0 - 6) (Sunday=0 or 7) OR sun,mon,tue,wed,th  
# | | | | |  
# | | | | |  
# * * * * * user-name command to be executed  
17 * * * * root cd / && run-parts --report /etc/cron.hourly  
25 6 * * * root test -x /usr/sbin/anacron || { cd / && run-parts --re  
port /etc/cron.daily; }  
47 6 * * 7 root test -x /usr/sbin/anacron || { cd / && run-parts --re  
port /etc/cron.weekly; }  
52 6 1 * * root test -x /usr/sbin/anacron || { cd / && run-parts --re  
port /etc/cron.monthly; }  
#
```

[+] Sudo Permissions

Matching Defaults entries for kali on kali:

```
env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/us  
r/sbin\:/usr/bin\:/sbin\:/bin, use_pty
```

User kali may run the following commands on kali:

```
(ALL : ALL) ALL
```

Scan Completed Successfully

ADVANTAGES

- Saves time by automating security checks
- Reduces manual auditing effort
- Detects real-world privilege escalation risks
- Ethical and safe (detection only)
- Easy to use with a single command
- Helps improve Linux system security
- Useful for learning and training purposes

LIMITATIONS

- Detection only – does not perform real exploitation
- Results depend on user permissions
- Works only on Linux environments
- Cannot detect zero-day vulnerabilities
- Requires manual review of findings

CONCLUSION

- The Linux Privilege Escalation Automation Toolkit automates the process of identifying privilege escalation risks in Linux systems.
- It helps security learners and professionals quickly detect misconfigurations and improve system security through structured reporting.

THANK YOU

THREAT INTELLIGENCE AGGREGATOR

Collects threats, finds risky ones, and creates blocklists for protection.

- Domain: Cybersecurity
- Technology: Kali Linux, Python 3
- Internship: Unified Mentor Internship – 2026
- Developed by: P.Hariharan UMID03122572891

THREAT INTELLIGENCE AGGREGATOR

A tool that collects cyber threat data, combines it, finds dangerous ones, and creates blocklists for protection.

Key Features

- Collects threat feeds from multiple sources
- Extracts IOCs (IP, domain, URL, hash, email)
- Cleans and standardizes data
- Detects repeated and high-risk threats
- Generates automated security blocklists
- Produces a summarized threat report

PROBLEM STATEMENT

- Threat data comes from multiple sources
- Feeds are in different formats (CSV, TXT, JSON, STIX)
- Manual analysis takes too much time
- Difficult to identify repeated threats
- No single unified threat list
- Slow creation of security blocklists

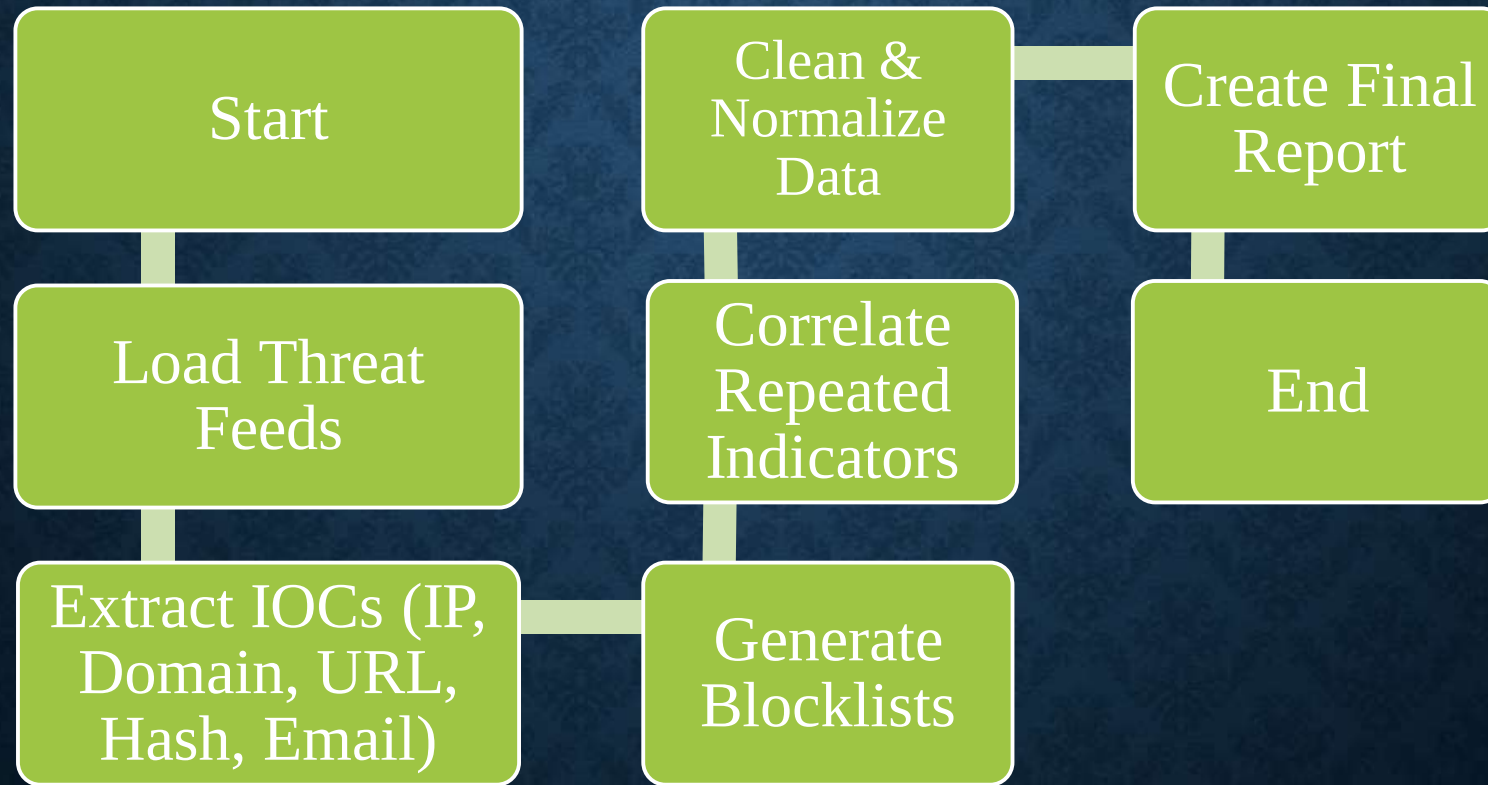
OBJECTIVES

- Collect threat intelligence from multiple feeds
- Extract IOCs (IP, domain, URL, hash, email)
- Normalize data into a unified format
- Identify repeated and high-risk indicators
- Generate blocklists for security tools
- Produce a final threat intelligence report

SYSTEM OVERVIEW

- Collects threat feeds from files or URLs
- Extracts malicious indicators (IOCs)
- Cleans and standardizes the data
- Identifies repeated and high-risk threats
- Generates blocklists for security tools
- Produces a final threat intelligence report

SYSTEM WORKFLOW



TOOLS AND TECHNOLOGIES

Programming Language

- Python

Libraries Used

- re – pattern matching (IOC extraction)
- requests – fetching online feeds
- json & csv – parsing feed data
- ipaddress – IP validation
- hashlib – hash verification

RESULT / OUTPUT

```
(kali@kali)-[~]  
$ cd Desktop  
  
(kali@kali)-[~/Desktop]  
$ cd feeds  
  
(kali@kali)-[~/Desktop/feeds]  
$ python3 aggregator.py  
  
[+] Parsing feeds ...  
[+] Total unique indicators: 25  
[+] Normalized indicators saved: output/normalized_iocs.csv  
[+] IP blocklist saved: output/IP_blocklist.txt  
[+] Domain blocklist saved: output/Domain_blocklist.txt  
[+] URL blocklist saved: output/URL_blocklist.txt  
[+] Hash blocklist saved: output/Hash_blocklist.txt  
[+] Email blocklist saved: output/Email_blocklist.txt  
[+] Summary report saved: output/report_summary.txt  
  
[+] Threat Intelligence Aggregation Complete!
```

CONCLUSION

- Developed a rule-based Threat Intelligence Aggregator
- Automated collection and processing of threat feeds
- Identified repeated and high-risk indicators
- Generated ready-to-use security blocklists
- Improved threat analysis and defensive security workflow

THANK YOU

PASSWORD CRACKING CREDENTIAL ATTACK SUITE

A tool that tests passwords against common wordlists to identify weak credentials.

- Domain: Cybersecurity
- Technology: Kali Linux, Python 3
- Internship: Unified Mentor Internship – 2026
- Developed by: P.Hariharan UMID03122572891

PASSWORD CRACKING CREDENTIAL ATTACK SUITE

This project demonstrates how dictionary attacks are used to identify weak passwords and improve authentication security.

Key Features

- Performs dictionary-based password cracking
- Uses real-world wordlists (RockYou)
- Detects weak passwords automatically
- Estimates cracking time
- Generates security assessment results

PROBLEM STATEMENT

- Many users create weak and common passwords
- Attackers can guess passwords using automated tools
- Poor password security leads to data breaches
- Organizations need tools to test password strength

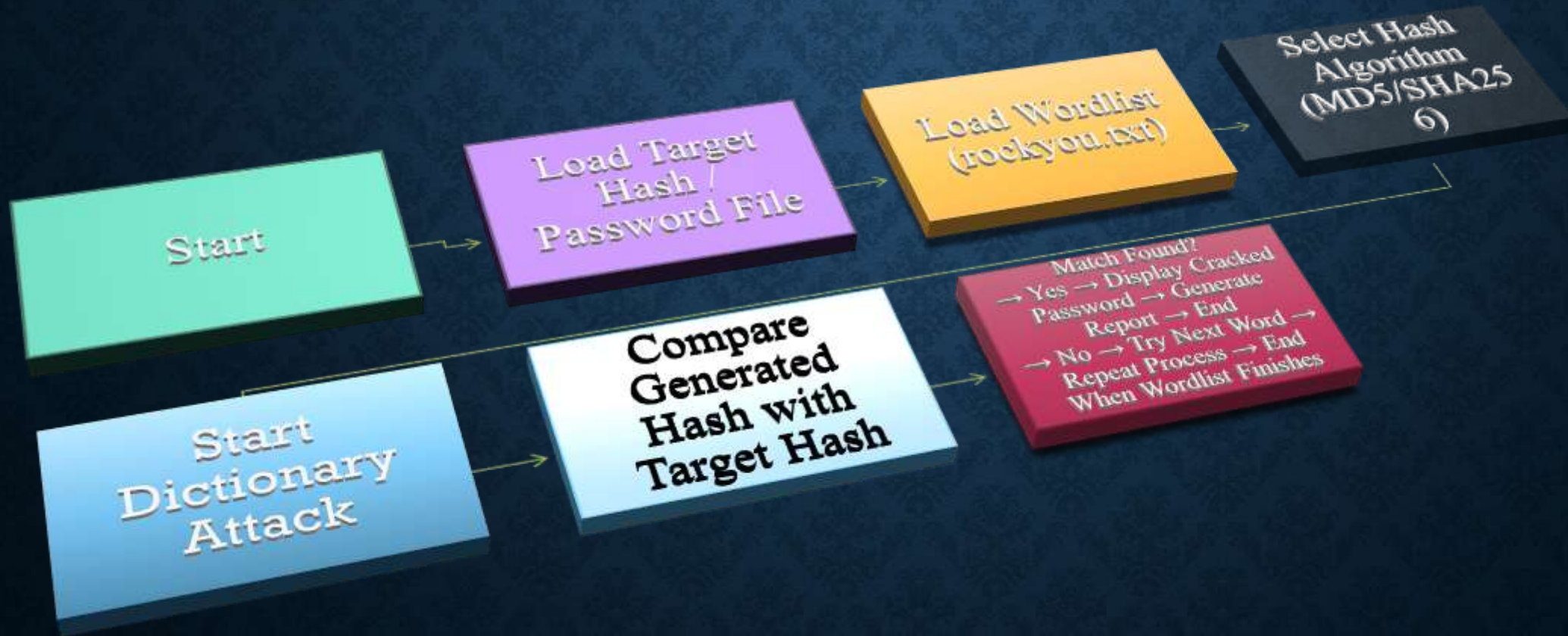
OBJECTIVES

- Demonstrate dictionary-based password cracking
- Identify weak and common passwords
- Estimate password cracking time
- Promote secure password practices
- Improve authentication security awareness

DICTIONARY ATTACK CONCEPT

- A password cracking method using a list of common passwords
- Tries each word from a wordlist automatically
- Based on the idea that users reuse simple passwords
- Faster than brute-force because it uses likely guesses
- Used in security testing to find weak passwords

SYSTEM WORKFLOW



TOOLS AND TECHNOLOGIES

- Kali Linux (Testing Environment)
- Python (Programming Language)
- RockYou Wordlist (Password Dictionary)
- Hashlib Library (Hash Generation)
- Terminal / Command Line (Execution)

IMPLEMENTATION IN KALI LINUX

- Create project folder and Python script
- Install and extract RockYou wordlist
- Generate target password hash
- Run the dictionary attack script
- Monitor cracking process in terminal
- Display cracked password and results

RESULT / OUTPUT

```
(kali@kali)-[~]
$ cd Desktop

(kali@kali)-[~/Desktop]
$ mkdir password_cracker_project

(kali@kali)-[~/Desktop]
$ cd password_cracker_project

(kali@kali)-[~/Desktop/password_cracker_project]
$ touch password_cracker.py

(kali@kali)-[~/Desktop/password_cracker_project]
$ touch target_password.txt

(kali@kali)-[~/Desktop/password_cracker_project]
$ nano target_password.txt

(kali@kali)-[~/Desktop/password_cracker_project]
$ nano password_cracker.py

(kali@kali)-[~/Desktop/password_cracker_project]
$ python password_cracker.py
=== PASSWORD CRACKER TOOL ===
Starting Dictionary Attack ...

Trying: 123456
Trying: 12345
Trying: 123456789
Trying: password
Trying: iloveyou
Trying: princess
Trying: 1234567
Trying: rockyou

PASSWORD FOUND: rockyou
```

WEAK PASSWORD DETECTION

- Identifies common and easily guessable passwords
- Flags short passwords and simple patterns
- Detects use of dictionary words and numbers
- Rates password strength (Low / Medium / High)
- Provides recommendations to create stronger passwords

ESTIMATED CRACKER TIME

- Calculates time based on wordlist size and speed
- Demonstrates how fast weak passwords can be cracked
- Shows difference between weak vs strong passwords
- Highlights importance of complex password creation

LEARNING OUTCOMES

- Understand password hashing and storage
- Learn ethical password testing techniques
- Perform authentication security auditing
- Compare red team vs blue team approaches
- Apply best practices for strong password creation

CONCLUSION

- Weak passwords are a major security risk
- Dictionary attacks can quickly expose poor password practices
- Security testing helps improve authentication systems
- Strong password policies reduce cyber attack risks

THANK YOU

USB DEVICE CONTROL MONITORING FRAMEWORK

A practical endpoint security project

- Domain: Cybersecurity
- Technology: Kali Linux, Python 3
- Internship: Unified Mentor Internship – 2026
- Developed by: P.Hariharan UMID03122572891

USB DEVICE CONTROL MONITORING FRAMEWORK

USB devices are widely used for data storage and transfer, but they also create serious security risks for computers and organizations.

Key Features:

- Real-time USB device detection
- Allowlist and Blocklist management
- Automatic blocking of unauthorized USB devices
- Monitoring plug-in and plug-out events
- File transfer auditing between system and USB drives
- Logging of all USB activities with timestamps
- Detection of suspicious or unknown devices
- Generation of final USB security audit report

PROBLEM STATEMENT

- USB devices are widely used but poorly monitored.
- Unauthorized USB usage can lead to data theft and malware infections.
- Organizations lack visibility into USB-based activities.
- Insider threats can copy sensitive data to external drives.
- Plug-and-play devices can introduce backdoors into systems.
- There is a need for a system to monitor, control, and audit USB usage.

OBJECTIVES

- Detect USB devices when they are connected or removed.
- Identify devices using Vendor ID and Product ID.
- Maintain an allowlist of trusted USB devices.
- Automatically block or unmount unauthorized USB devices.
- Monitor file transfers between system and USB drives.
- Log all USB activities with timestamps.
- Generate a complete USB security audit report.

SCOPE OF THE PROJECT

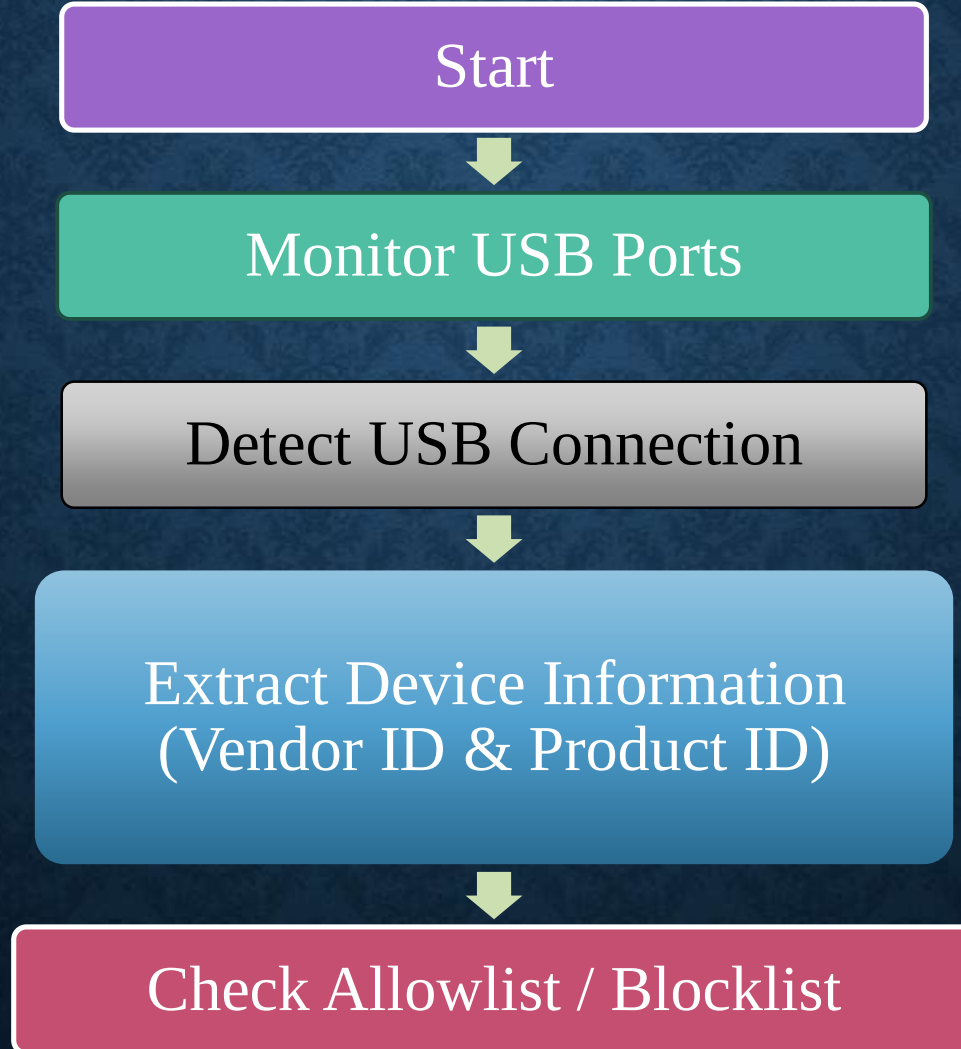
✓ Pros

- Provides real-time USB monitoring and detection
- Helps prevent unauthorized data copying
- Detects unknown or suspicious USB devices
- Generates audit logs for security analysis
- Improves endpoint security awareness
- Useful for academic and small-scale environments

✗ Cons

- Simulated blocking (not enterprise-level device control)
- Requires administrator/root privileges
- Limited to USB storage devices
- Cannot fully prevent advanced hardware spoofing
- Designed for Linux environment only

SYSTEM WORKFLOW





TOOLS AND TECHNOLOGIES

Operating System

- Kali Linux

Programming Language

- Python

• Python Libraries

- **pyudev** – USB device event monitoring
- **watchdog** – File transfer monitoring
- **json** – Allowlist & blocklist management

- **datetime** – Timestamp logging
- **os** – USB unmount automation

✂ **System Utilities**

- **lsusb** – USB device identification
- **udevadm** – Hardware event monitoring
- **VirtualBox** – Lab environment setup

IMPLEMENTATION IN KALI LINUX

- Installed required Python libraries (pyudev, watchdog)
- Created project workspace and configuration files
- Collected USB fingerprint (Vendor ID & Product ID) using **lsusb**
- Implemented real-time USB monitoring using Python + udev
- Developed auto-unmount script to block unauthorized USB devices
- Implemented file transfer monitoring for USB drives
- Generated logs and final USB security audit report

OUTPUT AND DATA RESULTS

```
(kali@kali)-[~]  
$ cd Desktop  
  
(kali@kali)-[~/Desktop]  
$ cd USB_Shield  
  
(kali@kali)-[~/Desktop/USB_Shield]  
$ sudo python3 usb_monitor.py  
USB Monitoring Started... Plug USB now  
/home/kali/Desktop/USB_Shield/usb_monitor.py:32: DeprecationWarning: Will be removed in 1.0. Access properties with Device.properties.  
    if usb_device:  
USB Connected → Vendor:03f0 Product:2d40  
🔴 Unauthorized USB Detected  
USB Connected → Vendor:03f0 Product:2d40  
🔴 Unauthorized USB Detected  
USB Connected → Vendor:03f0 Product:2d40  
🔴 Unauthorized USB Detected
```


CONCLUSION

- Successfully developed a USB Device Control & Monitoring Framework.
- Implemented real-time USB detection and authorization.
- Simulated automatic blocking of unauthorized USB devices.
- Monitored file transfers between system and USB drives.
- Generated detailed logs and audit reports.
- Gained practical experience in endpoint security and system monitoring.

FUTURE SCOPE

- Real enterprise-level USB blocking using kernel modules
- GUI dashboard for monitoring and alerts
- Email/SMS alert integration for security teams
- Support for Windows and macOS environments
- Detection of advanced USB spoofing attacks
- Integration with SIEM and Endpoint Detection & Response (EDR) systems